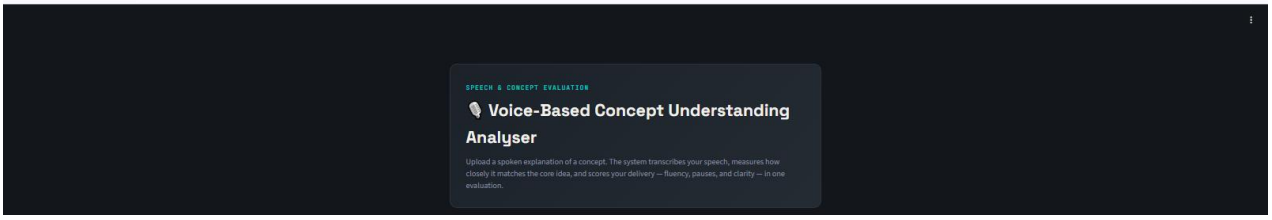
VBCUA: Voice-Based Concept Understanding Analyser

Project Description:

VBCUA is an AI-powered web application designed to evaluate how effectively users understand and explain conceptual topics through spoken communication. The platform combines speech-to-text transcription, semantic similarity analysis, audio feature extraction, and intelligent scoring mechanisms to assess both the quality of conceptual understanding and the fluency of speech delivery. Built using Streamlit and Python, the application delivers a responsive, interactive, and modular architecture that supports waveform visualization, AI-generated qualitative feedback, downloadable PDF reports, and structured evaluation suitable for students, educators, trainers, and researchers.

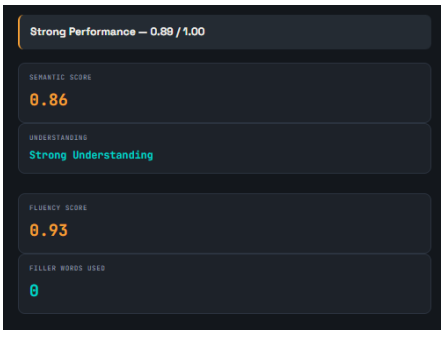


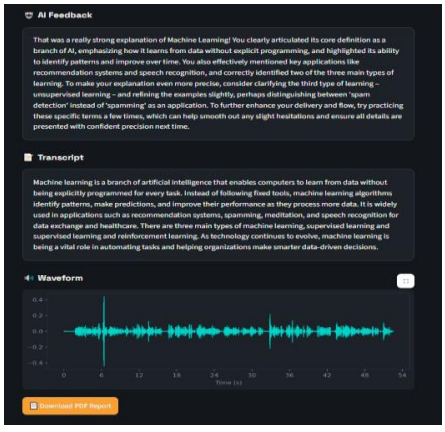
Scenarios:

Scenario 1: A student uploads an audio explanation for a concept such as "Machine Learning." The system transcribes the speech using OpenAI Whisper and compares the explanation against a predefined reference concept using Sentence-BERT semantic embeddings. The system generates a conceptual understanding score along with a qualitative label — Strong, Moderate, or Poor Understanding.

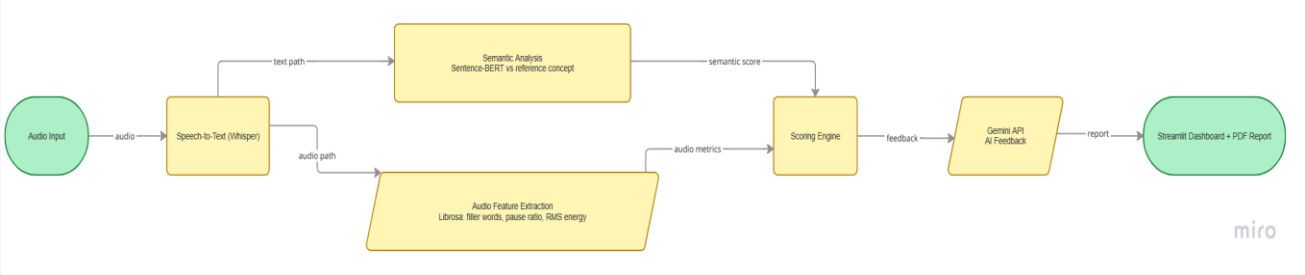
Scenario 2: A learner practicing interview preparation records a spoken explanation. The platform evaluates fluency-related metrics including filler word usage, pause ratio, and RMS energy using Librosa-based audio signal analysis, helping the user understand hesitation patterns and speaking clarity.

Scenario 3: A student reviews the generated evaluation report, including transcript, semantic similarity score, filler word statistics, waveform visualization, AI-generated feedback, and final score — all displayed in an interactive Streamlit dashboard, with the option to download a structured PDF report for future reference.





Technical Architecture:



Description:

VBCUA follows a modular pipeline architecture. The Streamlit frontend collects an audio file and a selected concept from the user, passing both to a backend processing pipeline. The pipeline sequentially calls four independent modules: a transcription module powered by OpenAI Whisper, a semantic analysis module powered by Sentence-BERT, an audio feature extraction module powered by Librosa, and a scoring module that merges both semantic and fluency outputs into a single final score. A Gemini API integration then converts the raw scores into a natural-language feedback paragraph. Results are rendered live in the Streamlit dashboard and can be exported as a structured PDF report containing the same information. The entire application is containerized using Docker and deployed publicly via Hugging Face Spaces.

Pre-requisites:

- Python Programming Proficiency: https://docs.python.org/3/
- Streamlit Framework Knowledge: https://docs.streamlit.io/
- Basic understanding of NLP concepts (embeddings, semantic similarity)
- Familiarity with audio processing concepts: https://librosa.org/doc/latest/index.html
- Google Generative AI (Gemini) API familiarity: https://ai.google.dev/gemini-api/docs
- Version Control with Git: https://git-scm.com/doc
- Docker basics for containerized deployment: https://docs.docker.com/

Project Workflow:

Milestone 1: Requirement Analysis & System Design

- Activity 1.1: Define the problem statement, target users, and core use cases
- Activity 1.2: Identify required technologies and define functional/non-functional requirements
- Activity 1.3: Design the system architecture, mapping the flow from audio input to final scored output

Milestone 2: Environment Setup & Initial Configuration

- Activity 2.1: Set up a Python virtual environment and install all required libraries
- Activity 2.2: Install and configure ffmpeg, a required system dependency for audio decoding
- Activity 2.3: Initialize the project folder structure, separating core logic into independent modules

Milestone 3: Core System Development

- Activity 3.1: Develop the transcription module using OpenAI Whisper
- Activity 3.2: Develop the semantic similarity module using Sentence-BERT
- Activity 3.3: Develop the audio feature extraction module using Librosa
- Activity 3.4: Develop the scoring engine combining semantic and fluency metrics

Milestone 4: Integration & Optimization

- Activity 4.1: Integrate all modules into a single pipeline function
- Activity 4.2: Integrate the Gemini API for AI-generated feedback
- Activity 4.3: Build the PDF report generator
- Activity 4.4: Design and apply a custom dark-themed UI

Milestone 5: Testing & Validation

- Activity 5.1: Conduct functional testing using real recorded audio
- Activity 5.2: Conduct performance testing and identify bottlenecks
- Activity 5.3: Implement retry logic for Gemini API rate-limiting

Milestone 6: Deployment & Conclusion

- Activity 6.1: Containerize and deploy via Hugging Face Spaces
- Activity 6.2: Resolve deployment-specific issues
- Activity 6.3: Document and finalize the project

Milestone 1: Requirement Analysis & System Design

In this milestone, the focus is on defining what VBCUA needs to do and how it should be structured before any code is written. This involves identifying the target users (students, educators, trainers preparing for interviews or presentations), and the core problem: there is no easily accessible tool that evaluates both *what* a person says and *how* they say it in one unified analysis.

Activity 1.1: Define the Problem Statement
The problem statement was developed using the "I am / I'm trying to / But / Because / Which makes me feel" framework:

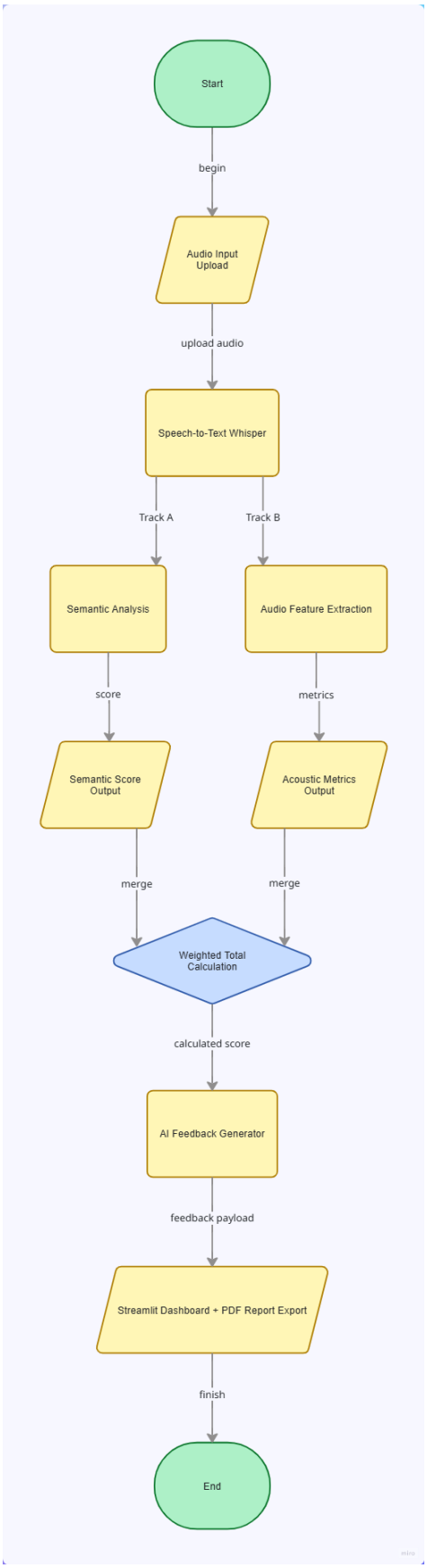
- I am a student preparing for interviews and presentations
- I'm trying to explain a concept clearly and confidently
- But I have no reliable way to know if my explanation was conceptually correct or well-delivered
- Because traditional feedback is unavailable, subjective, or too narrow
- Which makes me feel uncertain about my understanding and unable to track improvement

Activity 1.2: Identify Required Technologies
After reviewing the project requirements, the following technology stack was selected:

Category	Tool/Tech
Frontend/UI	Streamlit
Speech-to-Text	OpenAI Whisper
Semantic Analysis	Sentence-BERT (all-MiniLM-L6-v2)
Audio Processing	Librosa
AI Feedback	Google Gemini API (gemini-2.5-flash)
PDF Generation	fpdf2
Deployment	Docker + Hugging Face Spaces

Activity 1.3: Design the System Architecture

The architecture follows a linear pipeline: audio in → transcription → parallel semantic + audio analysis → combined scoring → AI feedback → display/export. This modular design allows each component to be developed and tested independently before integration.


Start
begin
Audio Input Upload
upload audio
Speech-to-Text Whisper
Track A
Track B
Semantic Analysis
Audio Feature Extraction
score
metrics
Semantic Score Output
Acoustic Metrics Output
merge
merge
Weighted Total Calculation
calculated score
AI Feedback Generator
feedback payload
Streamlit Dashboard + PDF Report Export
finish
End

Milestone 2: Environment Setup & Initial Configuration

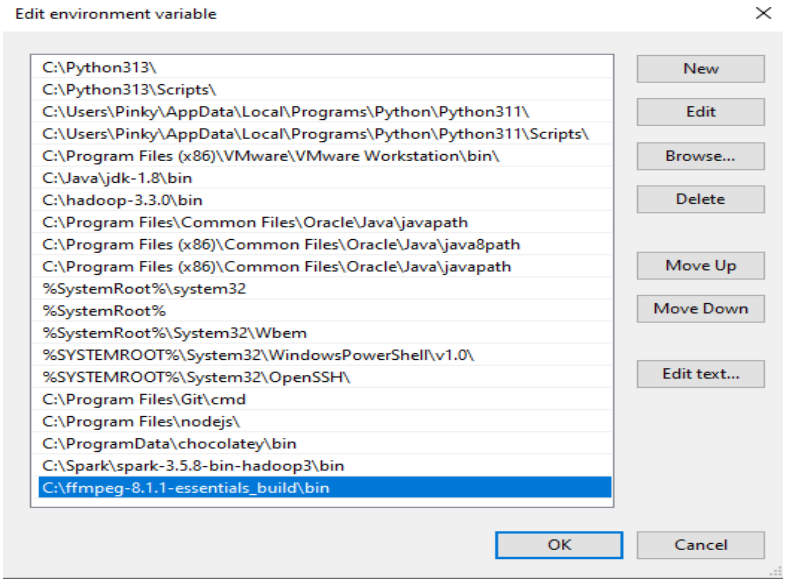
Activity 2.1: Python Environment and Dependency Installation

A virtual environment was created to isolate project dependencies:

```
python -m venv venv
venv\Scripts\activate
pip install streamlit librosa sentence-transformers openai-whisper torch numpy
matplotlib fpdf2 python-dotenv google-generativeai
```

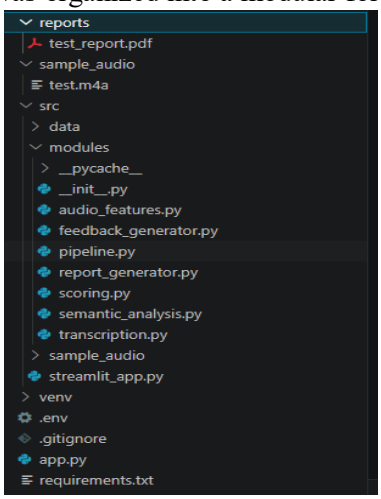
Activity 2.2: ffmpeg Installation

Whisper requires ffmpeg for audio decoding. This was downloaded, extracted, and added to the system PATH:



Activity 2.3: Project Structure Initialization

The project was organized into a modular folder structure:



Milestone 3: Core System Development

Activity 3.1: Speech-to-Text Module Development
The transcription module uses OpenAI Whisper's "base" model to convert spoken audio into text:

```python
import whisper
_model = whisper.load_model("base")

def transcribe_audio(audio_path: str) -> str:
    result = _model.transcribe(audio_path)
    return result["text"].strip()
```

Activity 3.2: Semantic Understanding & Similarity Engine
This module uses Sentence-BERT to compute semantic similarity between the user's transcript and a reference concept explanation:

```python
from sentence_transformers import SentenceTransformer, util
_model = SentenceTransformer("all-MiniLM-L6-v2")

def compute_similarity(transcript: str, reference_text: str) -> float:
    embeddings = _model.encode([transcript, reference_text], convert_to_tensor=True)
    return float(util.cos_sim(embeddings[0], embeddings[1])[0][0])
```

Activity 3.3: Audio Feature Extraction & Scoring Engine
This module uses Librosa to extract fluency-related metrics — filler word count, pause ratio, and RMS energy:

```python
import librosa
def analyze_audio(audio_path: str) -> dict:
    y, sr = librosa.load(audio_path, sr=16000)
    duration = librosa.get_duration(y=y, sr=sr)
    intervals = librosa.effects.split(y, top_db=30)
    voiced_duration = sum((end - start) for start, end in intervals) / sr
    pause_ratio = (duration - voiced_duration) / duration
    ...
```

Activity 3.4: Combined Scoring Engine
The final score merges semantic and fluency metrics using a weighted formula (60% understanding, 40% fluency):

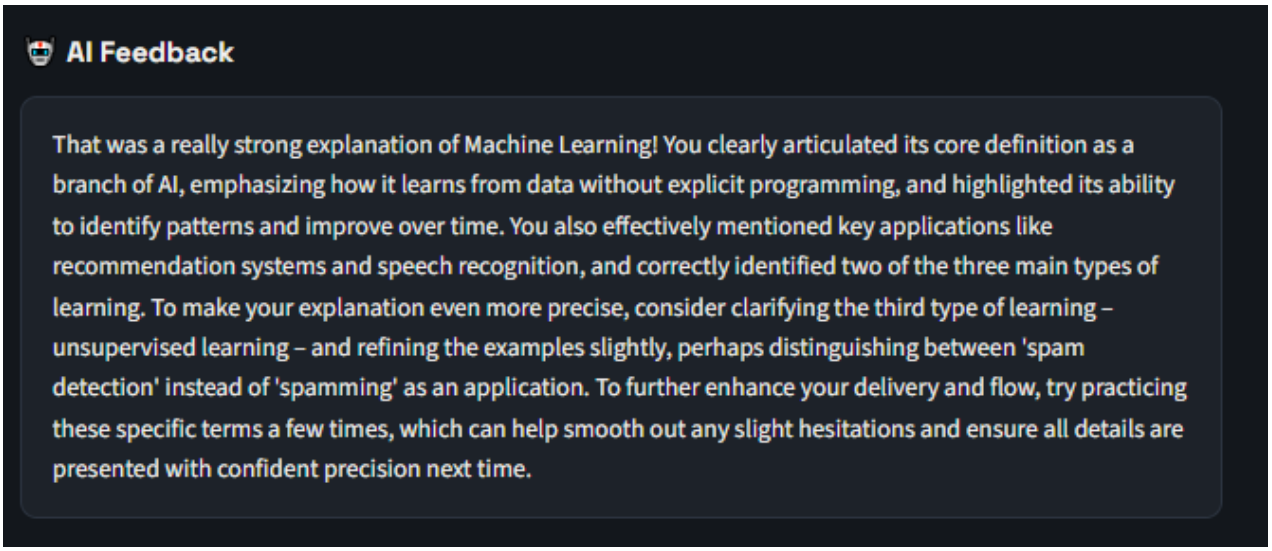
```python
def compute_final_score(semantic_score, filler_total, pause_ratio):
    filler_penalty = min(filler_total * 0.05, 0.3)
    pause_penalty = pause_ratio * 0.3
    fluency_score = max(1 - filler_penalty - pause_penalty, 0)
    final_score = (0.6 * semantic_score) + (0.4 * fluency_score)
    ...
```

Milestone 4: Integration & Optimization

Activity 4.1: Pipeline Integration
All modules were integrated into a single function, evaluate_explanation(), which takes an audio file and a concept name and returns the complete evaluation — transcript, scores, audio features, and final result — ready to be consumed by the UI.

Activity 4.2: AI Feedback Integration (Gemini API)
The Gemini API (gemini-2.5-flash model) was integrated to convert raw scores into natural, mentor-style feedback. Retry logic was added to gracefully handle free-tier rate limits:

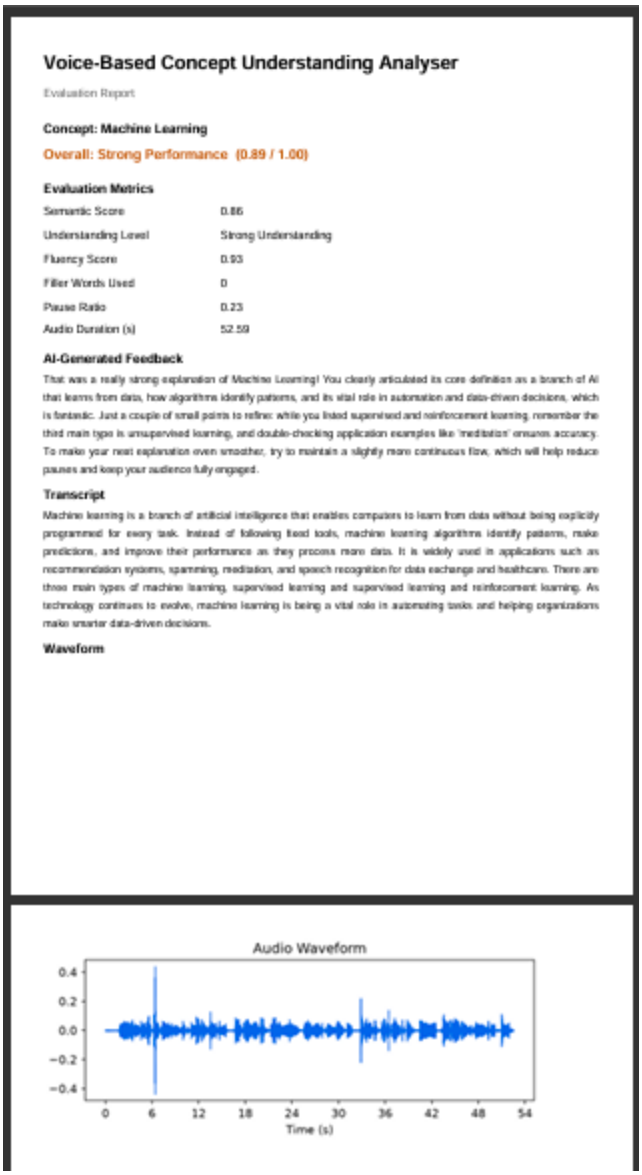
```python
for attempt in range(max_retries):
    try:
        response = _model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        if "429" in str(e) and attempt < max_retries - 1:
            time.sleep(10)
            continue
```




```
  import google.generativeai as genailable: 24.3.1 -> 26.1.2
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF
_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|
                      | 103/103 [00:00<00:00, 4370.26it/s]
D:\Gen-AI-Track-Project\venv\Lib\site-packages\whisper\transcribe.py:132: UserWa
rning: FP16 is not supported on CPU; using FP32 instead
  warnings.warn("FP16 is not supported on CPU; using FP32 instead")
D:\Gen-AI-Track-Project\modules\audio_features.py:28: UserWarning: PySoundFile f
ailed. Trying audioread instead.
  y, sr = librosa.load(audio_path, sr=16000)
D:\Gen-AI-Track-Project\venv\Lib\site-packages\librosa\core\audio.py:184: Future
Warning: librosa.core.audio.__audioread_load
        Deprecated as of librosa version 0.10.0.
        It will be removed in librosa version 1.0.
  y, sr_native = __audioread_load(path, offset, duration, dtype)
AI Feedback:
 That was a really strong explanation of Machine Learning! You beautifully captu
red its core definition as a branch of AI that learns from data without explicit
 programming, and you clearly articulated how algorithms identify patterns and i
mprove over time. You also effectively highlighted several key applications, lik
e recommendation systems and speech recognition, though you might want to clarif
y 'spamming' to 'spam detection' and double-check the 'meditation' example. Addi
tionally, while you correctly identified three types, remember the second one is
 *unsupervised* learning, alongside supervised and reinforcement learning. Overa
ll, your understanding is excellent; for even greater precision next time, try p
ausing briefly before listing examples or specific terms to ensure you articulat
e each one exactly as intended.
(venv) PS D:\Gen-AI-Track-Project>
```

Activity 4.3: PDF Report Generation

A PDF report generator was built using fpdf2, embedding the transcript, scores, AI feedback, and a rendered waveform image into a downloadable document.



Voice-Based Concept Understanding Analyser

Evaluation Report

Concept: Machine Learning

Overall: Strong Performance (0.89 / 1.00)

Evaluation Metrics

Semantic Score	0.86
Understanding Level	Strong Understanding
Fluency Score	0.93
Filler Words Used	0
Pause Ratio	0.23
Audio Duration (s)	52.59

AI-Generated Feedback

That was a really strong explanation of Machine Learning! You clearly articulated its core definition as a branch of AI that learns from data, how algorithms identify patterns, and its vital role in automation and data-driven decisions, which is fantastic. Just a couple of small points to refine: while you listed supervised and reinforcement learning, remember the third main type is unsupervised learning, and double-checking application examples like 'meditation' ensures accuracy. To make your next explanation even smoother, try to maintain a slightly more continuous flow, which will help reduce pauses and keep your audience fully engaged.

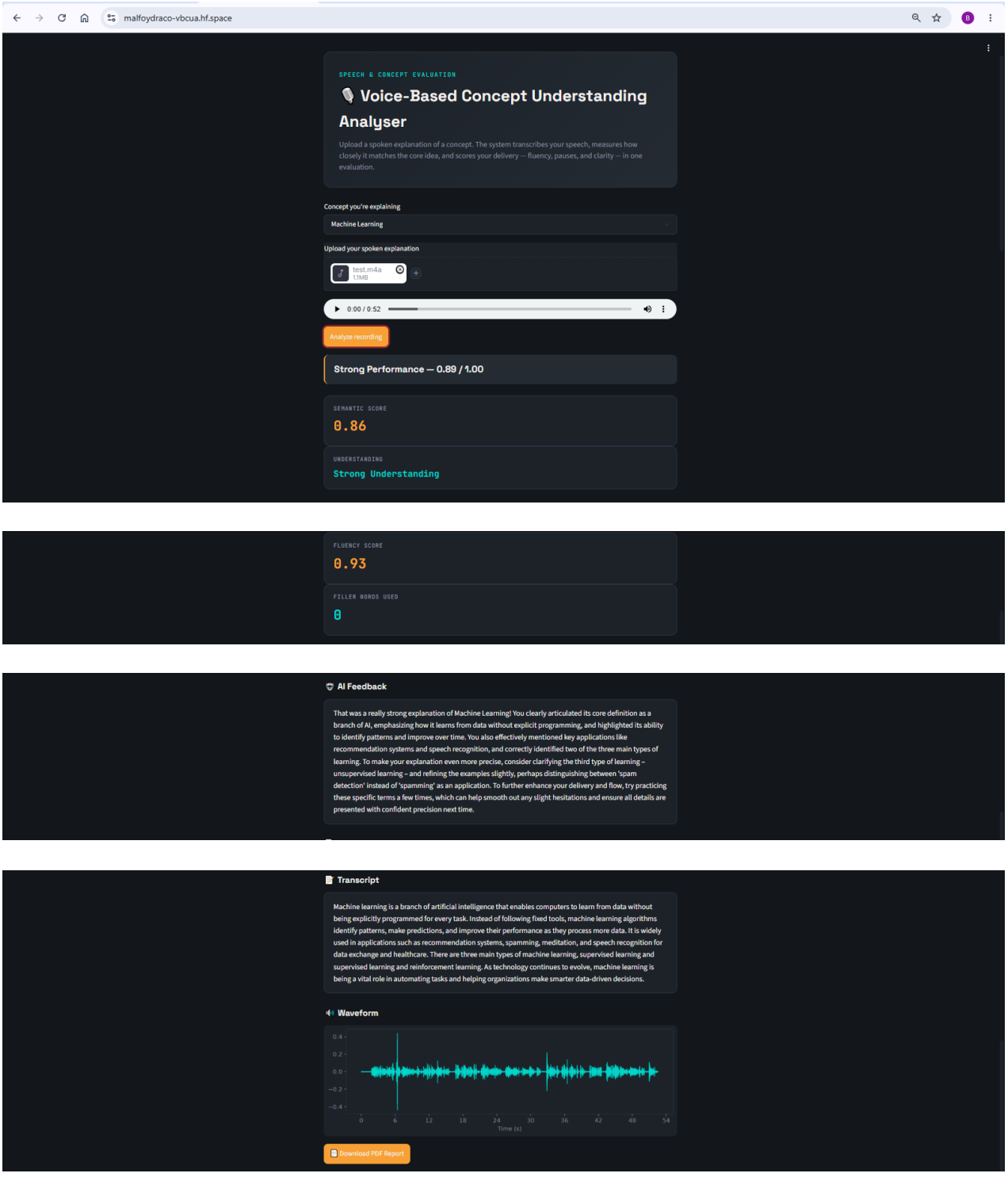
Transcript

Machine learning is a branch of artificial intelligence that enables computers to learn from data without being explicitly programmed for every task. Instead of following fixed tools, machine learning algorithms identify patterns, make predictions, and improve their performance as they process more data. It is widely used in applications such as recommendation systems, spamming, meditation, and speech recognition for data exchange and healthcare. There are three main types of machine learning, supervised learning and supervised learning and reinforcement learning. As technology continues to evolve, machine learning is being a vital role in automating tasks and helping organizations make smarter data-driven decisions.

Waveform

Activity 4.4: Custom UI Design

A custom dark "recording studio" theme was applied — charcoal background, amber and teal accents, Space Grotesk and JetBrains Mono typography — replacing Streamlit's default styling with a distinctive, purpose-built design.

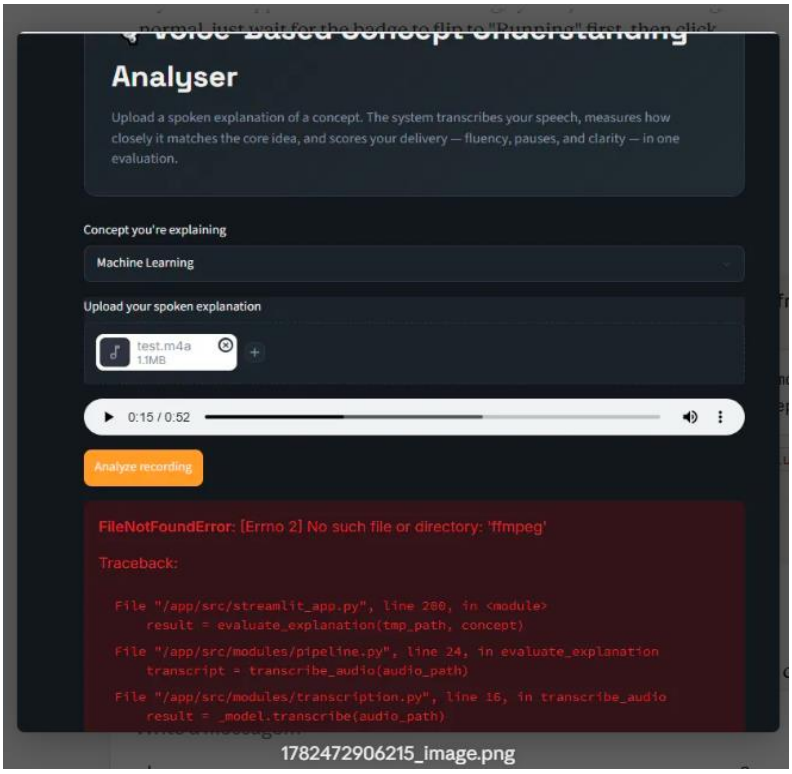

malfoydraco-vbcua.hf.space
SPEECH & CONCEPT EVALUATION
Voice-Based Concept Understanding Analyser
Upload a spoken explanation of a concept. The system transcribes your speech, measures how closely it matches the core idea, and scores your delivery — fluency, pauses, and clarity — in one evaluation.
Concept you're explaining
Machine Learning
Upload your spoken explanation
test.m4a
1.1MB
0:00 / 0:52
Analyze recording
Strong Performance — 0.89 / 1.00
SEMANTIC SCORE
0.86
UNDERSTANDING
Strong Understanding
FLUENCY SCORE
0.93
FILLER WORDS USED
0
AI Feedback
That was a really strong explanation of Machine Learning! You clearly articulated its core definition as a branch of AI, emphasizing how it learns from data without explicit programming, and highlighted its ability to identify patterns and improve over time. You also effectively mentioned key applications like recommendation systems and speech recognition, and correctly identified two of the three main types of learning. To make your explanation even more precise, consider clarifying the third type of learning – unsupervised learning – and refining the examples slightly, perhaps distinguishing between 'spam detection' instead of 'spamming' as an application. To further enhance your delivery and flow, try practicing these specific terms a few times, which can help smooth out any slight hesitations and ensure all details are presented with confident precision next time.
Transcript
Machine learning is a branch of artificial intelligence that enables computers to learn from data without being explicitly programmed for every task. Instead of following fixed tools, machine learning algorithms identify patterns, make predictions, and improve their performance as they process more data. It is widely used in applications such as recommendation systems, spamming, meditation, and speech recognition for data exchange and healthcare. There are three main types of machine learning, supervised learning and supervised learning and reinforcement learning. As technology continues to evolve, machine learning is being a vital role in automating tasks and helping organizations make smarter data-driven decisions.
Waveform
0.4
0.2
0.0
−0.2
−0.4
0
6
12
18
24
30
36
42
48
54
Time (s)
Download PDF Report

Milestone 5: Testing & Validation

Activity 5.1: Functional Testing
The application was tested end-to-end using real recorded audio across multiple concepts (Machine Learning, Cloud Computing), confirming accurate transcription, correct dynamic reference lookup, and consistent scoring behaviour.

Activity 5.2: Performance Testing
Response times were measured manually (~15-20 seconds per analysis). Bottlenecks identified included cold-start delays after Space inactivity and Gemini's free-tier rate limit (5 requests/minute).

Activity 5.3: Error Handling & Optimization
A genuine rate-limit error was encountered during testing and resolved by implementing automatic retry logic with a graceful fallback message.

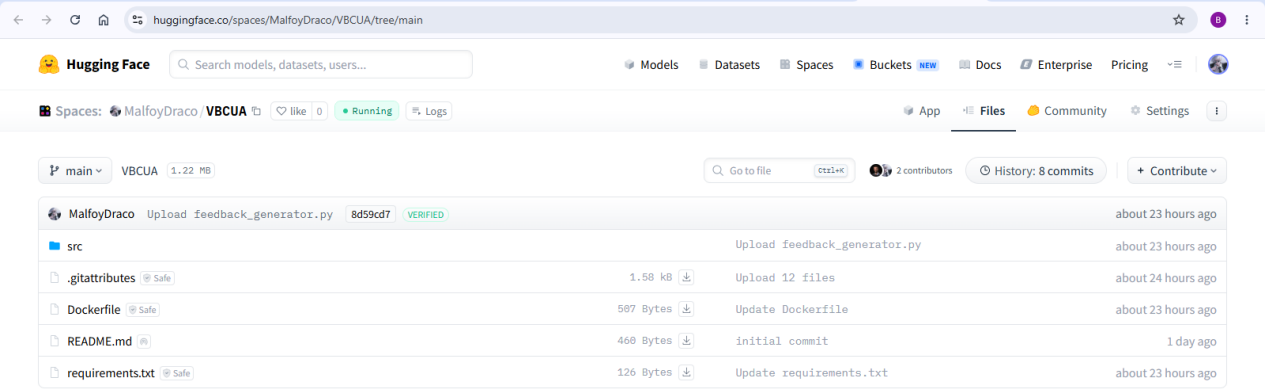


1782472906215_image.png

Milestone 6: Deployment & Conclusion

Activity 6.1: Containerization & Deployment

The application was containerized using Docker and deployed via Hugging Face Spaces. A Dockerfile was configured to copy source files, install dependencies, and launch the Streamlit server.



Activity 6.2: Deployment Issue Resolution

Several deployment-specific issues were identified and resolved:

- Missing ffmpeg in the Docker container, causing transcription failures — resolved by adding it to the Dockerfile's system dependencies
- Incorrect relative file paths for the reference concepts JSON — resolved using absolute paths based on the script's own location
- File upload 403 errors due to CORS/XSRF protection behind HF's proxy — resolved by disabling these flags in the Streamlit launch command

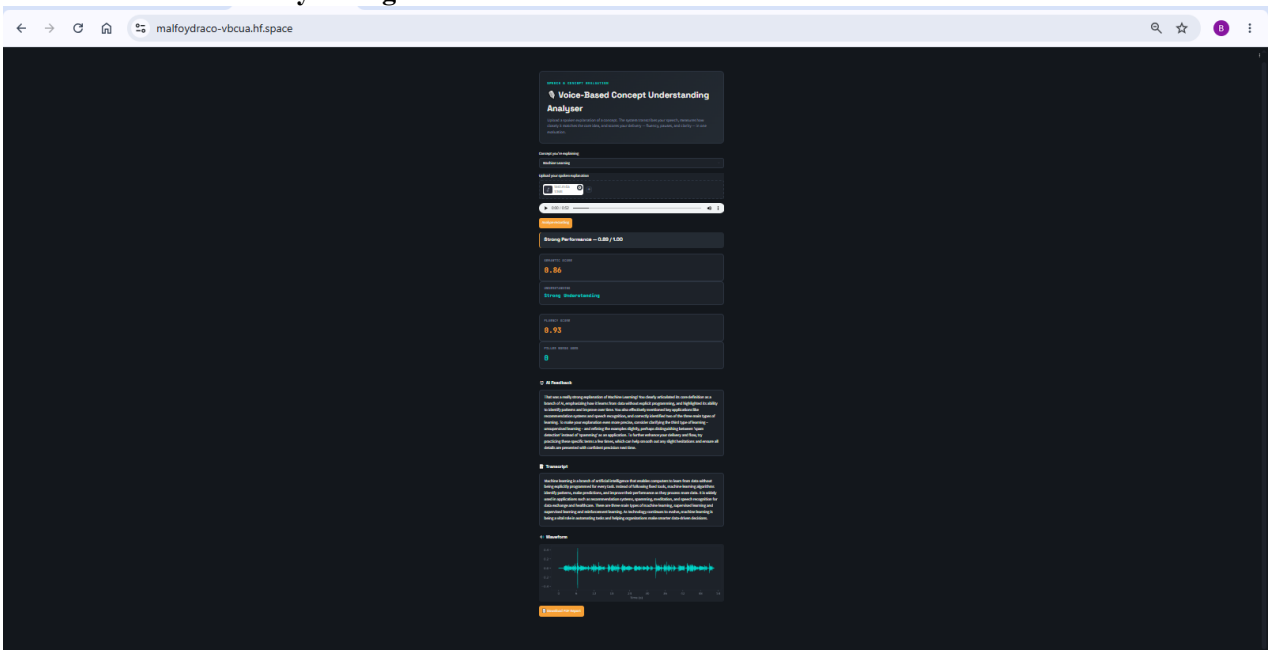
Activity 6.3: Final Public Link

The application is publicly accessible at: https://malfoydraco-vbcua.hf.space

```
https://malfoydraco-vbcua.hf.space
```

Exploring the Application's Pages

Main Dashboard / Analysis Page:



Description: The single-page interface of VBCUA serves as the entire application. It features a dark "recording studio" themed design with a hero header, a concept selection dropdown, and an audio upload component. Upon analysis, the page dynamically reveals a verdict banner, score cards, an AI-generated feedback section, the full transcript, a styled waveform visualization, and a PDF download button.

Conclusion:

VBCUA is a generative AI-powered platform that combines speech processing, semantic NLP, and audio signal analysis to provide an automated, objective evaluation of spoken conceptual explanations. By integrating OpenAI Whisper, Sentence-BERT, Librosa, and the Gemini API within a Streamlit interface, the application delivers fast, structured feedback covering both *what* a user understands and *how* they communicate it. The project, which included independent module development, full pipeline integration, and public deployment via Hugging Face Spaces, demonstrates how multiple AI techniques can be combined into a single cohesive educational tool. Looking ahead, VBCUA has strong potential for expansion, including support for additional concepts, multi-language transcription, personalized progress tracking, and integration with classroom/LMS platforms.